

Práctica: Game Loop

Autores	Rol	Porcentaje
André Jesús Gómez Linares	Elaboración del guion y exposición	100%
André Jesús Gómez Linares	Elaboración de ejemplos de código	100%
Total		100%

Entregables	URL
Repositorio	https://github.com/agomezl-maker/the_Gameloop
Diapositivas	https://prezi.com/p/edit/_lv2t-qlmn6m/

Game Loop

El Game Loop es fundamental en el desarrollo de videojuegos, ya que es el mecanismo que permite que el mundo del juego se mantenga activo de forma continua, sincronizando la entrada del jugador, la lógica interna y el dibujado en pantalla. Sin él, un videojuego no podría comportarse de manera interactiva ni estable, lo que lo convierte en un componente esencial de cualquier motor de juego.

Diapositivas de la exposición: Ver presentación en Prezi

¿Qué es el Game Loop?

El Game Loop es el corazón de cualquier videojuego: conecta y sincroniza todos sus sistemas. A diferencia del software tradicional, que es reactivo y permanece inactivo hasta que el usuario realiza una acción, un videojuego necesita seguir funcionando incluso si el jugador no toca nada (enemigos pensando su siguiente movimiento, efectos visuales corriendo, tiempo avanzando). Por eso se necesita un bucle que nunca se detenga.

Importancia del Game Loop

- Mantiene el mundo del juego activo en todo momento.
- Sincroniza entrada, lógica y renderizado.
- Garantiza una experiencia interactiva en tiempo real.
- Permite controlar el rendimiento de forma independiente del hardware.
- Es la base de cualquier motor de videojuego profesional.

Estructura Básica del Game Loop

El ciclo se repite idealmente cada 16.6 milisegundos (60 imágenes por segundo) y ejecuta en orden tres fases:

Fase	Descripción
Input	Se revisa de inmediato qué está haciendo el jugador (teclado, control, etc.).
Update	Se calcula la lógica y la física del juego en memoria RAM: posiciones, colisiones, vida, enemigos.
Render	Se borra el frame anterior y se dibuja en pantalla el nuevo estado del mundo.

Este ciclo Input → Update → Render se repite sin parar desde que se abre el juego hasta que se cierra.

Implementación en Java

El bucle infinito no puede ejecutarse en el hilo principal, ya que congelaría toda la ventana del sistema. Por eso es obligatorio crear un hilo secundario, usando la clase **Thread** y la interfaz **Runnable**, para que el Game Loop corra de forma independiente.

El problema del hardware

Un bucle simple, sin control de tiempo, depende directamente de la potencia de la computadora: en una máquina rápida el bucle se ejecuta miles de veces por segundo, provocando movimiento absurdamente rápido e injugable; en una máquina lenta, el juego corre extremadamente lento. En un desarrollo profesional esto es inaceptable.

Separación de UPS y FPS

Usando `System.nanoTime()` (precisión en nanosegundos) se separan dos conceptos:

Concepto	Descripción
UPS (Updates Per Second)	Actualizaciones de lógica por segundo. Deben mantenerse fijos (p. ej. 60 UPS).
FPS (Frames Per Second)	Cuadros dibujados por segundo. Pueden variar según la tarjeta gráfica.

Fixed Timestep

La técnica de Fixed Timestep (paso de tiempo fijo) acumula en una variable el tiempo transcurrido desde la última actualización. Cuando el acumulador alcanza el umbral necesario (16.66 ms para 60 UPS), se ejecuta una actualización de lógica y se resta ese tiempo del acumulador. El renderizado, en cambio, se ejecuta tan rápido como el hardware lo permita. Así, aunque los gráficos pierdan calidad o velocidad en una máquina más débil, la física y el movimiento del juego se mantienen siempre estables.

Ejemplo: Esqueleto del Game Loop

```
while (juego activo) {
    Input(); // Leer entrada del jugador
```

```
// Fixed Timestep para Update
while (acumulador >= 16.66ms) {
    Update(); // Logica y fisica a UPS fijos
    acumulador -= 16.66ms;
}

Render(); // Dibujar a la velocidad que el hardware permita (FPS variable)
}
```

Analogía relacionada al tema

El Game Loop puede compararse con un director de orquesta: así como el director guía a los músicos para que toquen en sincronía, el Game Loop guía a las funciones y sistemas del juego (entrada, lógica y dibujado) para que trabajen en armonía y produzcan el resultado deseado.

Conclusiones

El Game Loop no es solo un fragmento de código más: es el motor de vida de cualquier videojuego. Controlar el tiempo con precisión usando nanosegundos y manejar correctamente los hilos de ejecución es lo que marca la diferencia entre un proyecto amateur, con caídas de rendimiento y comportamientos inconsistentes, y un sistema profesional, fluido y estable. Dominar esta estructura de tiempo garantiza una buena experiencia de usuario, sin importar en qué computadora se esté jugando.